

CS 6160: Introduction to Cryptology

Programming Assignment 2

Short Report

Name: Shubham Kumar
Roll No.- cs26mtech12012

- The Machine & Processor Used:

Specification	Value
Machine	HP Pavilion Laptop 15-eg2xxx
Operating System	Ubuntu (64-bit)
Architecture	x86_64
Processor	12th Gen Intel Core i7-1260P
Vendor	Intel (GenuineIntel)
Physical Cores	12
Logical CPUs	16
Threads per Core	2
Cores / Threads	12 cores, 16 threads
Maximum CPU Frequency	4.70 GHz
RAM	16 GB (14 GiB total, ~7 GiB available)
L1 Cache	448 KiB (Data), 640 KiB (Instruction)
L2 Cache	9 MiB
L3 Cache	18 MiB
AES-NI Support	Yes (aes flag present)

- Compiler and Compiler Flags:

Compiler: g++ (Ubuntu 15.2.0)
C++ Standard: C++17
Optimization Level: -O3
AES-NI Flags: -maes -msse4.1
Compilation Command: g++ -std=c++17 -O3 -maes -msse4.1 Q1.cpp -o Q1

● Description of the Implementation:

I have written the implementation in C++ consisting of two versions of the same AES-like block cipher: a conventional software implementation and a hardware-accelerated implementation using AES-NI intrinsics. Both implementations use the same modified key schedule and produce identical ciphertexts for the same plaintext and key.

1. Modified Key Schedule

The standard AES key schedule is not used. Instead, a simplified key schedule is defined as:

- The master key K is used directly as the pre-whitening key: $k_{-1} = K$
- Each subsequent round key is derived recursively: $k_i = \text{ShiftRows}(k_{i-1}) \oplus \text{RC}_i$ for $i = 0, 1, \dots, 9$
- RC_i is the i -th round constant (01, 02, 04, 08, 10, 20, 40, 80, 1B, 36), XORed into the first byte only
- This produces 11 round keys in total: $k_{-1}, k_0, k_1, \dots, k_9$

2. Software Implementation

The software implementation explicitly performs all AES transformations on a 16-byte state in column-major order. The following components were implemented manually:

- SubBytes : Byte substitution using the standard AES S-Box
- ShiftRows : Bytes permuted using the fixed order {0, 5, 10, 15, 4, 9, 14, 3, 8, 13, 2, 7, 12, 1, 6, 11}
- MixColumns : Finite-field multiplication in $\text{GF}(2^8)$ using the `xtime()` helper function
- AddRoundKey : Bitwise XOR of the state with the round key
- Inverse operations : InvSubBytes (AES inverse S-Box), InvShiftRows, InvMixColumns using `mul9`, `mul11`, `mul13`, `mul14` helpers

Encryption sequence:

1. AddRoundKey with k_{-1}
2. Rounds 1 to 9: SubBytes $\rightarrow$ ShiftRows $\rightarrow$ MixColumns $\rightarrow$ AddRoundKey
3. Final round: SubBytes $\rightarrow$ ShiftRows $\rightarrow$ AddRoundKey with k_9 (no MixColumns)

Decryption sequence:

1. AddRoundKey with k_9
2. Rounds 9 to 1: InvShiftRows $\rightarrow$ InvSubBytes $\rightarrow$ AddRoundKey $\rightarrow$ InvMixColumns
3. Final step: InvShiftRows $\rightarrow$ InvSubBytes $\rightarrow$ AddRoundKey with k_{-1}

3. AES-NI Implementation

The hardware-accelerated implementation uses Intel AES-NI intrinsics from <immintrin.h>:

- Key generation : Uses `_mm_shuffle_epi8()` for ShiftRows on 128-bit registers and `_mm_xor_si128()` to XOR the round constants
- Encryption : Uses `_mm_aesenc_si128()` for rounds 1 to 9 (performs SubBytes → ShiftRows → MixColumns → AddRoundKey in one instruction) and `_mm_aesenc_si128()` for the final round
- Decryption keys : Middle round keys are transformed using `_mm_aesimc_si128()` to apply Inverse MixColumns
- Decryption : Uses `_mm_aesdec_si128()` for rounds 1 to 9 and `_mm_aesdec_si128()` for the final round

4. Performance Measurement

- Throughput measured in MB/s as N/T where N = bytes processed, T = time in seconds using `chrono::high_resolution_clock`
- Cycles per byte (CPB) measured using the `__rdtsc()` timestamp counter
- Key generation is excluded from all timing measurements
- Each experiment repeated 20 times and the average is reported

• Correctness Verification

Before benchmarking, correctness was verified by:

1. Encrypting and decrypting a fixed plaintext and confirming the recovered plaintext matches the original
2. Confirming that software and AES-NI produce identical ciphertexts for the same key and plaintext
3. Running 1000 randomized tests with random keys and plaintexts — all 1000/1000 passed

```

shubham@shubham-HP-Pavilion-Laptop-15-eg2xxx:~/Code/CryptologyProgrammingAssignment2$ ./Trail
===== SOFTWARE AES =====

Master Key:
00 01 02 03 04 05 06 07 08 09 0a 0b 0c 0d 0e 0f

Plaintext:
00 11 22 33 44 55 66 77 88 99 aa bb cc dd ee ff

Ciphertext:
17 45 e7 88 59 24 6d 57 39 c9 34 15 3a 79 aa c8

Recovered Plaintext:
00 11 22 33 44 55 66 77 88 99 aa bb cc dd ee ff

Software AES: SUCCESS

===== AES-NI =====

AES-NI Ciphertext:
17 45 e7 88 59 24 6d 57 39 c9 34 15 3a 79 aa c8

Software and AES-NI ciphertext MATCH.

AES-NI Recovered Plaintext:
00 11 22 33 44 55 66 77 88 99 aa bb cc dd ee ff

AES-NI Decryption: SUCCESS

=====
Randomized Correctness Test: 1000/1000 PASSED

```

- Throughput Results (MB/s)

Implementation	1 KB	2 KB	4 KB	32 KB	64 KB
Software Encryption	16.20	25.08	31.17	67.88	81.71
AES-NI Encryption	2870.55	4265.40	5210.42	12756.14	13706.29
Software Decryption	14.11	25.29	26.36	65.73	66.35
AES-NI Decryption	2724.78	4415.34	6106.38	13704.34	14205.51

- Cycles per Byte (CPB)

Implementation	1 KB	2 KB	4 KB	32 KB	64 KB
Software Encryption	146.55	94.70	76.32	35.03	29.13
AES-NI Encryption	0.65	0.50	0.44	0.19	0.17
Software Decryption	168.52	94.05	90.27	36.21	35.87
AES-NI Decryption	0.68	0.47	0.37	0.17	0.17

```
shubham@shubham-HP-Pavilion-Laptop-15-eg2xxx:~/Code/CryptologyProgrammingAssignment2$ ./Trail
=====
PERFORMANCE BENCHMARK (Average of 20 Runs)
=====
```

Size	Implementation	Throughput (MB/s)	CPB
1 KB	SW Encrypt	16.20	146.55
1 KB	AES Encrypt	2870.55	0.65
1 KB	SW Decrypt	14.11	168.52
1 KB	AES Decrypt	2724.78	0.68
2 KB	SW Encrypt	25.08	94.70
2 KB	AES Encrypt	4265.40	0.50
2 KB	SW Decrypt	25.29	94.05
2 KB	AES Decrypt	4415.34	0.47
4 KB	SW Encrypt	31.17	76.32
4 KB	AES Encrypt	5210.42	0.44
4 KB	SW Decrypt	26.36	90.27
4 KB	AES Decrypt	6106.38	0.37
32 KB	SW Encrypt	67.88	35.03
32 KB	AES Encrypt	12756.14	0.19
32 KB	SW Decrypt	65.73	36.21
32 KB	AES Decrypt	13704.34	0.17
64 KB	SW Encrypt	81.71	29.13
64 KB	AES Encrypt	13706.29	0.17
64 KB	SW Decrypt	66.35	35.87
64 KB	AES Decrypt	14205.51	0.17

● Brief Discussion of the Results

1. How much faster is AES-NI encryption compared with your software implementation?

AES-NI encryption was significantly faster than the conventional software implementation for all tested input sizes. The speedup increased as the message size became larger because the overhead of function calls and timing became less significant.

Input Size	Software (MB/s)	AES-NI (MB/s)	Speedup
1 KB	16.20	2870.55	177.19×
2 KB	25.08	4265.40	170.07×
4 KB	31.17	5210.42	167.16×
32 KB	67.88	12756.14	187.92×
64 KB	81.71	13706.29	167.74×

At 64 KB, AES-NI achieved 13.71 GB/s, compared with 81.71 MB/s for the software implementation.

2. How much faster is AES-NI decryption compared with your software implementation?

AES-NI decryption also showed a substantial performance improvement over software decryption.

Input Size	Software (MB/s)	AES-NI (MB/s)	Speedup
------------	-----------------	---------------	---------

1 KB	14.11	2724.78	193.11×
2 KB	25.29	4415.34	174.58×
4 KB	26.36	6106.38	231.65×
32 KB	65.73	13704.34	208.49×
64 KB	66.35	14205.51	214.10×

The highest measured improvement was approximately 214× for the 64 KB input.

3. Is the performance improvement similar for encryption and decryption? If not, why might they differ?

Both encryption and decryption showed performance improvement with AES-NI, although their performance gains were not exactly the same. For larger message sizes, such as 32 KB and 64 KB, AES-NI decryption was slightly faster than AES-NI encryption. At 64 KB, for example, encryption achieved 13,706.29 MB/s, while decryption reached 14,205.51 MB/s.

The difference is quite small and is most likely due to normal hardware-level factors such as processor scheduling, cache usage, pipeline efficiency, and minor variations during benchmarking. Since both operations rely on dedicated AES hardware instructions ([AESENC](#) for encryption and [AESDEC](#) for decryption), their overall performance remains very similar.

4. Compare the throughput and cycles per byte obtained for encryption and decryption.

The benchmark shows that as throughput increases, the number of CPU cycles needed for each byte decreases. In simple terms, the program processes data faster while using fewer CPU cycles per byte.

For the software implementation, encryption speed increased from 16.20 MB/s for a 1 KB message to 81.71 MB/s for a 64 KB message. At the same time, the cycles per byte dropped from 146.55 to 29.13. Software decryption also became faster, improving from 14.11 MB/s to 66.35 MB/s, while its cycles per byte decreased from 168.52 to 35.87.

The AES-NI implementation performed much better. Encryption speed increased from 2,870.55 MB/s to 13,706.29 MB/s, and the cycles per byte dropped from 0.65 to 0.17. AES-NI decryption showed similar results, reaching 14,205.51 MB/s at 64 KB while using only 0.17 cycles per byte.

5. How do the measured results change as the message size increases?

As the message size increased from 1 KB to 64 KB, the performance improved for both implementations. The throughput became higher, and the number of cycles per byte (CPB) became lower.

This happens because the extra work involved in function calls, timing, and loop setup is spread across more data. Larger inputs also make better use of the CPU cache and processor pipeline, which helps the program run more efficiently.

The improvement was much more noticeable with AES-NI. Its encryption speed increased from about 2.87 GB/s at 1 KB to 13.71 GB/s at 64 KB, while the CPB dropped from 0.65 to 0.17. The software implementation also became faster with larger inputs, but the improvement was smaller because it performs every AES operation using software instructions instead of dedicated hardware instructions.